

Git and github document

1) Basic Git Commands (For managing repositories)

—>**git init**: Initializes a new Git repository in the current directory.

```
waseemakram@waseem myproject.git % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /Users/waseemakram/myproject.git/.git/
```

—> **git clone <repo_url>**: Clones a repository from a remote URL to your local machine.

```
waseemakram@waseem myproject.git % git clone git@github.com:Waseema761/git-document.git
Cloning into 'git-document'...
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 0), reused 4 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (4/4), done.
waseemakram@waseem myproject.git % ls
bmw      git-document  world
waseemakram@waseem myproject.git %
```

git status: Shows the working directory status (which files are modified, staged, etc.).

```
waseemakram@waseem myproject.git % git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        bmw
        world

nothing added to commit but untracked files present (use "git add" to track)
```

git add : Stages a file or files to be committed

git commit -m "message": Commits staged changes to the local repository with a message.

```
waseemakram@waseem myproject.git % git add .
waseemakram@waseem myproject.git % git commit -m " added two file "
[master (root-commit) 874fc01] added two file
2 files changed, 1 insertion(+)
create mode 100644 bmw
create mode 100644 world
```

git push: Pushes committed changes to the remote repository.

It pushes changes to branch

```
[waseemakram@waseem myproject.git % git push -u origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 263 bytes | 263.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Waseema761/git-document.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.
```

git pull: Fetches and merges changes from the remote

repository to your local working directory.

It pull all the latest changes from central repo

```
[waseemakram@waseem myproject.git % git pull
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 2 (delta 0), reused 2 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (2/2), 254 bytes | 63.00 KiB/s, done.
From github.com:Waseema761/git-document
  874fc01..bf5dd50  master    -> origin/master
Updating 874fc01..bf5dd50
Fast-forward
 test | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 test
[waseemakram@waseem myproject.git % ls
bmw          git-document  test          world
```

git fetch: Downloads objects and refs from another repository (without merging).

git merge : Merges changes from the specified branch into the current branch.

```
[waseemakram@waseem myproject.git % git merge release
Updating 2d53ec2..7e6c580
Fast-forward
 file1 | 0
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 file1
[waseemakram@waseem myproject.git % ls
bmw          file1          git-document  test          world
[waseemakram@waseem myproject.git %
```

git log: Shows the commit history of the repository.

```
waseemakram@waseem myproject.git % git log
commit 7e6c5808bed7ab49696303669e54a64315167706 (HEAD -> master, origin/release, release)
Merge: f30fdcd 2d53ec2
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:17:10 2025 +0530

    Merge branch 'master' into release

commit f30fdcd48db42f3beecee07e57fd99245f285aab
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:16:46 2025 +0530

    added file1

commit 2d53ec2ad96fe67fad34e255ee219b3f64b018e6 (origin/master, origin/HEAD)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:12:14 2025 +0530

    added clone of this repo

commit bf5dd505934b9d19ded3cadd01edd7fa44f0b598
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 12:36:16 2025 +0530

    added test from clone
```

git diff: Shows the differences between your working directory and the index (staged changes).

```
added two files
waseemakram@waseem myproject.git % git diff release origin/master
diff --git a/file1 b/file1
deleted file mode 100644
index e69de29..0000000
waseemakram@waseem myproject.git % git diff
waseemakram@waseem myproject.git % git diff 7e6c58 f30fdc
diff --git a/git-document b/git-document
deleted file mode 160000
index bf5dd50..0000000
--- a/git-document
+++ /dev/null
@@ -1,0,0 @@
-Subproject commit bf5dd505934b9d19ded3cadd01edd7fa44f0b598
waseemakram@waseem myproject.git %
```

2. Branching Commands (Managing branches)

`git branch`: Lists all branches in your repository.

```
[waseemakram@waseem myproject.git % git branch
* master
  release
waseemakram@waseem myproject.git %
```

`git branch <branch_name>`:
Creates a new branch

```
waseemakram@waseem myproject.git % git fetch
[waseemakram@waseem myproject.git % git branch release
[waseemakram@waseem myproject.git % git status
On branch master
```

`git checkout <branch_name>`: Switches to an existing branch.

```
[waseemakram@waseem myproject.git % git branch
* master
  release
[waseemakram@waseem myproject.git % git checkout release
warning: unable to rmdir 'git-document': Directory not empty
Switched to branch 'release'
```

`git checkout -b <branch_name>`: Creates and switches to a new branch in one step.

```
release
[waseemakram@waseem myproject.git % git checkout -b bmw
Switched to a new branch 'bmw'
[waseemakram@waseem myproject.git % git branch
* bmw
  master
  release
waseemakram@waseem myproject.git %
```

`git rebase <branch_name>`: Applies changes from one branch onto another branch (instead of merging).

```
create mode 100644 cat
[waseemakram@waseem myproject.git % git rebase master
Current branch bmw is up to date.
[waseemakram@waseem myproject.git % git rebase release
Current branch bmw is up to date.
waseemakram@waseem myproject.git %
```

`git branch -d <branch_name>`: Deletes the specified branch (locally).

`git branch -D <branch_name>`: Forcefully deletes a branch (locally).

```
[waseemakram@waseem myproject.git % git branch -d bmw
error: the branch 'bmw' is not fully merged
hint: If you are sure you want to delete it, run 'git branch -D bmw'
hint: Disable this message with "git config set advice.forceDeleteBranch false"
[waseemakram@waseem myproject.git % git branch
bmw
master
* release
[waseemakram@waseem myproject.git % git branch -D bmw
Deleted branch bmw (was 5d1b66e).
waseemakram@waseem myproject.git %
```

`git remote set-head origin <branch_name>`: Sets the default branch for the remote repository.

```
waseemakram@waseem myproject.git % git remote set-head origin master
waseemakram@waseem myproject.git % git remote show origin

* remote origin
Fetch URL: git@github.com:Waseema761/git-document.git
Push URL: git@github.com:Waseema761/git-document.git
HEAD branch: master
Remote branches:
  master tracked
  release tracked
Local branches configured for 'git pull':
  master merges with remote master
  release merges with remote release
Local refs configured for 'git push':
  master pushes to master (fast-forwardable)
  release pushes to release (up to date)
waseemakram@waseem myproject.git %
```

3. Remote Commands (Managing remotes and syncing with the cloud)

`git remote -v`: Shows the remote repository URLs.

```
waseemakram@waseem myproject.git % git remote -v
origin git@github.com:Waseema761/git-document.git (fetch)
origin git@github.com:Waseema761/git-document.git (push)
waseemakram@waseem myproject.git %
```

`git remote add` : Adds a new remote repository.

```
waseemakram@waseem myproject.git % git remote add origin git@github.com:Waseema761/git-document.git
waseemakram@waseem myproject.git % ls
```

`git remote remove` : Removes a remote repository.

`git push origin <branch_name>`: Pushes a branch to a remote repository.

```
waseemakram@waseem git-document % git push -u origin master
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 8 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 274 bytes | 274.00 KiB/s, done.
Total 2 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Waseema761/git-document.git
   874fc01..bf5dd50  master -> master
branch 'master' set up to track 'origin/master'.
waseemakram@waseem git-document %
```

`git remote show origin`: Displays detailed information about a remote repository.

```
waseemakram@waseem myproject.git % git remote show origin master
* remote origin
Fetch URL: git@github.com:Waseema761/git-document.git
Push URL: git@github.com:Waseema761/git-document.git
HEAD branch: master
Remote branches:
  master tracked
  release tracked
Local branches configured for 'git pull':
  master merges with remote master
  release merges with remote release
Local refs configured for 'git push':
  master pushes to master (fast-forwardable)
  release pushes to release (up to date)
fatal: 'master' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
waseemakram@waseem myproject.git %
```

`git fetch origin`: Downloads objects and refs from the remote repository.

```
waseemakram@waseem myproject.git % git fetch origin master  
  
From github.com:Waseema761/git-document  
* branch          master    -> FETCH_HEAD
```

`git push origin <branch_name>`: Pushes a branch to a remote repository.

```
waseemakram@waseem myproject.git % git push  
Everything up-to-date  
waseemakram@waseem myproject.git % git push origin release  
Everything up-to-date  
waseemakram@waseem myproject.git % git pull  
Already up to date.  
waseemakram@waseem myproject.git % git pull origin release  
From github.com:Waseema761/git-document  
* branch          release    -> FETCH_HEAD  
Already up to date.  
waseemakram@waseem myproject.git %
```

|

`git pull origin <branch_name>`: Fetches and merges a branch from the remote repository.

`git remote show origin`: Displays detailed information about a remote repository

```
[waseemakram@waseem myproject.git % git remote show origin master
* remote origin
Fetch URL: git@github.com:Waseema761/git-document.git
Push URL: git@github.com:Waseema761/git-document.git
HEAD branch: master
Remote branches:
  master tracked
  release tracked
Local branches configured for 'git pull':
  master merges with remote master
  release merges with remote release
Local refs configured for 'git push':
  master pushes to master (fast-forwardable)
  release pushes to release (up to date)
fatal: 'master' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
```

4. Staging and Committing (Managing changes)

`git add .` : Stages all modified files in the working directory.

```
[waseemakram@waseem myproject.git % git add .
[waseemakram@waseem myproject.git % git status
On branch bmw
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        deleted:    git-document
```

`git add -A`: Stages all modified files (including file deletions).

```
and the repository exists.  
[waseemakram@waseem myproject.git % git add -A  
[waseemakram@waseem myproject.git % git status  
On branch release  
Your branch is up to date with 'origin/release'.  
  
nothing to commit, working tree clean  
waseemakram@waseem myproject.git %
```

`git reset` : Unstages a file (reverts it to the working directory).

```
[waseemakram@waseem myproject.git % git status  
On branch release  
Your branch is up to date with 'origin/release'.  
  
Changes to be committed:  
  (use "git restore --staged <file>..." to unstage)  
    new file:   hello  
  
[waseemakram@waseem myproject.git % git reset  
[waseemakram@waseem myproject.git % git status  
On branch release  
Your branch is up to date with 'origin/release'.  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    hello  
  
nothing added to commit but untracked files present (use "git add" to track)  
waseemakram@waseem myproject.git %
```

`git commit --amend`: Modifies the last commit (can change the commit message or add changes).

```
Merge branch 'master' into release

# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# Date:      Tue Nov 18 15:17:10 2025 +0530
#
# On branch release
# Your branch and 'origin/release' have diverged,
# and have 1 and 1 different commits each, respectively.
#
# Changes to be committed:
#   new file:   git-document
#
# Untracked files:
#   hello
#
~
~
~
~
```

```
[waseemakram@waseem myproject.git % git commit --amend
[release 8c3ea5e] Merge branch 'master' into release
Date: Tue Nov 18 15:17:10 2025 +0530
waseemakram@waseem myproject.git %
```

`git commit --all`: Automatically stages tracked files and commits them.

```
[waseemakram@waseem myproject.git % git commit --all
On branch release
Your branch and 'origin/release' have diverged,
and have 2 and 1 different commits each, respectively.

nothing to commit, working tree clean
waseemakram@waseem myproject.git %
```

git commit -a: Stages and commits all modified files.

```
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch release
# Your branch and 'origin/release' have diverged,
# and have 2 and 1 different commits each, respectively.
#
# Changes to be committed:
#   modified:   hello
#
~
~
~
~
~
~
~
```

```
[waseemakram@waseem myproject.git % git commit -a
[release 7bace72] modefied hello
1 file changed, 1 insertion(+)
[waseemakram@waseem myproject.git % git status
```

git reset --hard HEAD~1: Moves the HEAD pointer back one commit and discards changes in the working directory.

```
[waseemakram@waseem myproject.git % git reset --hard HEAD~1
HEAD is now at f0235ba hello
[waseemakram@waseem myproject.git % git log -2
commit f0235bad1d7bfabc3b626532e643cae0fff09cec (HEAD -> release)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 16:33:39 2025 +0530

hello
```

`git commit --no-verify`: Skips pre-commit hooks while committing.

```
deleted hello
# Please enter the commit message for your changes. Lines starting
# with '#' will be ignored, and an empty message aborts the commit.
#
# On branch release
# Your branch and 'origin/release' have diverged,
# and have 3 and 1 different commits each, respectively.
#
# Changes to be committed:
#   deleted:    hello
#
~
~
~
```

```
waseemakram@waseem myproject.git % git add .
[waseemakram@waseem myproject.git % git commit --no-verify
[release abff1e5] deleted hello
1 file changed, 1 deletion(-)
delete mode 100644 hello
waseemakram@waseem myproject.git %
```

`git reset --soft HEAD~1`: Moves the HEAD pointer back one commit but leaves your changes staged.

```
[waseemakram@waseem myproject.git % git log -2
commit abff1e565d35722aef9d455c399c74d807eb2c0c (HEAD -> release)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 17:06:45 2025 +0530

    deleted hello

commit 7bace72d6ef8a06a70a010aec57145893aea3139
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 17:03:36 2025 +0530

    modified hello
[waseemakram@waseem myproject.git % git reset --soft HEAD~1
[waseemakram@waseem myproject.git % git log -2
commit 7bace72d6ef8a06a70a010aec57145893aea3139 (HEAD -> release)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 17:03:36 2025 +0530

    modified hello

commit f0235bad1d7bfabc3b626532e643cae0fff09cec
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 16:33:39 2025 +0530

    hello
[waseemakram@waseem myproject.git %
```

5. Viewing and Comparing Changes (Inspecting changes)

`git diff`: Shows changes between your working directory and the index (unstaged changes).

```
[waseemakram@waseem myproject.git % git diff release origin/master
diff --git a/file1 b/file1
deleted file mode 100644
index e69de29..0000000
[waseemakram@waseem myproject.git % git diff
[waseemakram@waseem myproject.git % git diff 7e6c58 f30fdc
diff --git a/git-document b/git-document
deleted file mode 160000
index bf5dd50..0000000
--- a/git-document
+++ /dev/null
@@ -1,0,0 @@
-Subproject commit bf5dd505934b9d19ded3cadd01edd7fa44f0b598
```

`git diff --staged`: Shows changes between the index (staged files) and the last commit.

```
[waseemakram@waseem myproject.git % vi hello
[waseemakram@waseem myproject.git % git add .
[waseemakram@waseem myproject.git % git diff --staged
diff --git a/hello b/hello
index e69de29..ce01362 100644
--- a/hello
+++ b/hello
@@ -0,0 +1 @@
+hello
```

`git diff <commit_id>`: Shows the differences between your working directory and a specific commit.

```
[waseemakram@waseem myproject.git % git diff f0235b
diff --git a/hello b/hello
index e69de29..ce01362 100644
--- a/hello
+++ b/hello
@@ -0,0 +1 @@
+hello
waseemakram@waseem myproject.git %
```

`git log`: Displays the commit history.

```
[waseemakram@waseem myproject.git % git log -2
commit f0235bad1d7bfabc3b626532e643cae0fff09cec (HEAD -> release)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 16:33:39 2025 +0530

    hello

commit 8c3ea5e547b0f83db4f6ebcef4e98e8454c9019e
Merge: f30fdcd 2d53ec2
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:17:10 2025 +0530

    Merge branch 'master' into release
[waseemakram@waseem myproject.git % git status
```

`git log --oneline`: Displays the commit history in a simplified one-line format.

`git log --graph`: Displays the commit history as a graph.

```
hello
[waseemakram@waseem myproject.git % git log --oneline
f0235ba (HEAD -> release) hello
8c3ea5e Merge branch 'master' into release
f30fdcd added file1
2d53ec2 (origin/master, origin/HEAD) added clone of this repo
bf5dd50 added test from clone
874fc01 added two file
[waseemakram@waseem myproject.git % git log --graph
* commit f0235bad1d7bfabc3b626532e643cae0fff09cec (HEAD -> release)
| Author: waseem_akram <waseem.akram082003@gmail.com>
| Date: Tue Nov 18 16:33:39 2025 +0530
|
| hello
|
* commit 8c3ea5e547b0f83db4f6ebcef4e98e8454c9019e
| Merge: f30fdcd 2d53ec2
| Author: waseem_akram <waseem.akram082003@gmail.com>
| Date: Tue Nov 18 15:17:10 2025 +0530
|
| Merge branch 'master' into release
|
* commit 2d53ec2ad96fe67fad34e255ee219b3f64b018e6 (origin/master, origin/HEAD)
| Author: waseem_akram <waseem.akram082003@gmail.com>
| Date: Tue Nov 18 15:12:14 2025 +0530
|
| added clone of this repo
|
* commit f30fdcd48db42f3beecee07e57fd99245f285aab
| Author: waseem_akram <waseem.akram082003@gmail.com>
| Date: Tue Nov 18 15:16:46 2025 +0530
|
| added file1
|
* commit bf5dd505934b9d19ded3cadd01edd7fa44f0b598
| Author: waseem_akram <waseem.akram082003@gmail.com>
| Date: Tue Nov 18 12:36:16 2025 +0530
|
| added test from clone
```

`git log --author="name"`: Filters commits by a specific author.

```
added two file
[waseemakram@waseem myproject.git % git log --author='waseem'
commit f0235bad1d7bfabc3b626532e643cae0fff09cec (HEAD -> release)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 16:33:39 2025 +0530

hello

commit 8c3ea5e547b0f83db4f6ebcef4e98e8454c9019e
Merge: f30fdcd 2d53ec2
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:17:10 2025 +0530

Merge branch 'master' into release
```

`git log --since="2 weeks ago"`: Filters commits made after a specific date.

```
[waseemakram@waseem myproject.git % git log --since="2 weeks ago"
commit f0235bad1d7bfabc3b626532e643cae0fff09cec (HEAD -> release)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 16:33:39 2025 +0530

hello

commit 8c3ea5e547b0f83db4f6ebcef4e98e8454c9019e
Merge: f30fdcd 2d53ec2
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:17:10 2025 +0530

Merge branch 'master' into release

commit f30fdcd48db42f3beecee07e57fd99245f285aab
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:16:46 2025 +0530

added file1

commit 2d53ec2ad96fe67fad34e255ee219b3f64b018e6 (origin/master, origin/HEAD)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:12:14 2025 +0530

added clone of this repo

commit bf5dd505934b9d19ded3cadd01edd7fa44f0b598
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 12:36:16 2025 +0530

added test from clone
```

git blame : Shows line-by-line annotations for a file, telling who last modified each line.

```
[waseemakram@waseem myproject.git % git blame
usage: git blame [<options>] [<rev-opts>] [<rev>] [--] <file>

<rev-opts> are documented in git-rev-list(1)

--[no-]incremental    show blame entries as we find them, incrementally
-b                   do not show object names of boundary commits (Default: off)
--[no-]root           do not treat root commits as boundaries (Default: off)
--[no-]show-stats      show work cost statistics
--[no-]progress        force progress reporting
--[no-]score-debug     show output score for blame entries
-f, --[no-]show-name  show original filename (Default: auto)
-n, --[no-]show-number show original linenumber (Default: off)
-p, --[no-]porcelain  show in a format designed for machine consumption
--[no-]line-porcelain show porcelain format with per-line commit information
-c                   use the same output mode as git-annotate (Default: off)
-t                   show raw timestamp (Default: off)
-l                   show long commit SHA1 (Default: off)
-s                   suppress author name and timestamp (Default: off)
-e, --[no-]show-email show author email instead of name (Default: off)
-w                   ignore whitespace differences
--[no-]ignore-rev <rev>
                    ignore <rev> when blaming
--[no-]ignore-revs-file <file>
                    ignore revisions from <file>
--[no-]color-lines     color redundant metadata from previous line differently
--[no-]color-by-age    color lines by age
--[no-]minimal         spend extra cycles to find better match
-S <file>             use revisions from <file> instead of calling git-rev-list
--[no-]contents <file>
                    use <file>'s contents as the final image
-C[<score>]           find line copies within and across files
-M[<score>]           find line movements within and across files
-L <range>            process only line range <start>,<end> or function :<funcname>
--[no-]abbrev[=<n>]   use <n> digits to display object names
```

`git show <commit_id>`: Shows details of a specific commit (including diff and metadata).

```
[waseemakram@waseem myproject.git % git show f0235b
commit f0235bad1d7bfabc3b626532e643cae0fff09cec (HEAD -> release)
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 16:33:39 2025 +0530

    hello

diff --git a/hello b/hello
new file mode 100644
index 0000000..e69de29
waseemakram@waseem myproject.git %
```

6. Reverting and Resetting (Undoing changes)

`git reset <commit_id>`: Moves HEAD to a specific commit, leaving your working directory intact (can be used with `--soft`, `--mixed`, or `--hard`).

```
[waseemakram@waseem myproject.git % git reset f0235b
Unstaged changes after reset:
M    hello
[waseemakram@waseem myproject.git % git log --online
```

`git reset --hard <commit_id>`: Resets your working directory, index, and HEAD to a specific commit (all changes will be lost).

`git reset --soft <commit_id>`: Resets only the HEAD to a specific commit, leaving staged changes.

```
[waseemakram@waseem myproject.git % git reset --hard f0235ba
HEAD is now at f0235ba  hello
[waseemakram@waseem myproject.git % git reset --soft f0235ba
[waseemakram@waseem myproject.git % git log --oneline
f0235ba (HEAD -> release)  hello
8c3ea5e Merge branch 'master' into release
f30fdcd  added file1
2d53ec2 (origin/master, origin/HEAD)  added clone of this repo
bf5dd50  added test from clone
874fc01  added two file
waseemakram@waseem myproject.git %
```

`git reset --mixed <commit_id>`: Resets the HEAD and index to a specific commit but keeps the working directory unchanged.

```
874fc01  added two file
[waseemakram@waseem myproject.git % git reset --mixed 8c3ea5e
[waseemakram@waseem myproject.git % git status
On branch release
Your branch and 'origin/release' have diverged,
and have 1 and 1 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hello

nothing added to commit but untracked files present (use "git add" to track)
waseemakram@waseem myproject.git %
```

`git revert <commit_id>`: Creates a new commit that undoes the changes of the specified commit.

```
fatal: revert failed
[waseemakram@waseem myproject.git % git revert f30fdcd
[release 077088f] Revert " added file1"
 1 file changed, 0 insertions(+), 0 deletions(-)
 delete mode 100644 file1
waseemakram@waseem myproject.git %
```

`git clean -f`: Removes untracked files in the working directory.

`git clean -fd`: Removes untracked files and directories.

```
waseemakram@waseem myproject.git % git clean -f
Removing fil3
waseemakram@waseem myproject.git %
```

7. Stashing (Temporarily saving changes)

`git stash`: Stashes the changes in your working directory (both staged and unstaged).

`git stash list`: Lists all the stashes

```
waseemakram@waseem myproject.git % git stash
Saved working directory and index state WIP on release: 42473ef hello
```

```
waseemakram@waseem myproject.git % git stash list
stash@{0}: WIP on release: 42473ef hello
waseemakram@waseem myproject.git %
```

git stash pop: Applies the latest stash and removes it from the stash list.

```
stash@{0}: WIP on release: 42473ef hello
[waseemakram@waseem myproject.git % git stash pop
On branch release
Your branch and 'origin/release' have diverged,
and have 3 and 1 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   fil4

Dropped refs/stash@{0} (e61caca5586cb49fa40dcf1d1442ffe6be55e4d2)
waseemakram@waseem myproject.git %
```

git stash apply: Applies a stash without removing it from the list.

```
[waseemakram@waseem myproject.git % git stash
Saved working directory and index state WIP on release: 42473ef hello
[waseemakram@waseem myproject.git % git stash apply
On branch release
Your branch and 'origin/release' have diverged,
and have 3 and 1 different commits each, respectively.
(use "git pull" if you want to integrate the remote branch with yours)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   fil4

waseemakram@waseem myproject.git %
```

git stash drop: Removes a specific stash from the stash list.

```
[waseemakram@waseem myproject.git % git stash list
stash@{0}: WIP on release: 42473ef hello
[waseemakram@waseem myproject.git % git stash drop
Dropped refs/stash@{0} (f83261ef6f497cd5ae4094d6b018e635e6a65ef7)
waseemakram@waseem myproject.git %
```

git stash clear: Clears all stashes.

git stash save "message": Stashes changes with an optional message for identification.

```
waseemakram@waseem myproject.git % git add .  
waseemakram@waseem myproject.git % git stash save " hello"  
Saved working directory and index state On release:  hello  
waseemakram@waseem myproject.git % git stash list  
stash@{0}: On release: hello  
waseemakram@waseem myproject.git %
```

git stash branch <branch_name>: Creates a new branch from the stash and applies it.

```
stash@{0}: On release: hello  
[waseemakram@waseem myproject.git % git stash branch car  
Switched to a new branch 'car'  
On branch car  
Changes to be committed:  
  (use "git restore --staged <file>..." to unstage)  
    new file:   file4  
  
Dropped refs/stash@{0} (0a12b8bc344695b7509776d68fff516cca1af9e7)  
[waseemakram@waseem myproject.git % git branch  
* car  
  master  
  release  
waseemakram@waseem myproject.git %
```

8. Tags (Managing versions)

`git tag`: Lists all tags in the repository

`git tag <tag_name>`: Creates a new tag at the current commit.

`git tag -a <tag_name> -m "message"`: Creates an annotated tag with a message.

```
waseemakram@waseem myproject.git % git tag v1.0
waseemakram@waseem myproject.git % git log -1
commit 7e6c5808bed7ab49696303669e54a64315167706 (HEAD -> master, tag: v1.0, origin/release)
Merge: f30fdcd 2d53ec2
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:17:10 2025 +0530

Merge branch 'master' into release
waseemakram@waseem myproject.git % git tag -a v1.0 -m " hi there "
fatal: tag 'v1.0' already exists
waseemakram@waseem myproject.git % git tag -a v2.0 -m " hi there "
waseemakram@waseem myproject.git % git log -1
commit 7e6c5808bed7ab49696303669e54a64315167706 (HEAD -> master, tag: v2.0, tag: v1.0, origin/release)
Merge: f30fdcd 2d53ec2
Author: waseem_akram <waseem.akram082003@gmail.com>
Date: Tue Nov 18 15:17:10 2025 +0530

Merge branch 'master' into release
waseemakram@waseem myproject.git %
```

`git push origin <tag_name>`: Pushes a specific tag to the remote repository.

`git push origin --tags`: Pushes all tags to the remote repository

```
Merge branch 'master' into release
waseemakram@waseem myproject.git % git push origin v1.0
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Waseema761/git-document.git
 * [new tag]          v1.0 -> v1.0
waseemakram@waseem myproject.git % git push origin --tags
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 163 bytes | 163.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To github.com:Waseema761/git-document.git
 * [new tag]          v2.0 -> v2.0
waseemakram@waseem myproject.git %
```

9. Git Config and Info (Configuration and repository information)

`git config --global user.name "Your Name"`: Sets your global username for commits.

```
waseemakram@waseem myproject.git % git config --global user.name waseem_akram
waseemakram@waseem myproject.git % git config --global user.name " waseem-akram"
waseemakram@waseem myproject.git % git config --global user.name waseem-akram
waseemakram@waseem myproject.git %
```

`git config --global user.email "youremail@example.com"`: Sets your global email for commits.

```
waseem-akram
waseemakram@waseem myproject.git % git config --global user.email waseem.akram082003@gmail.com
waseemakram@waseem myproject.git %
```

`git config --list`: Lists all configuration settings.

`git config` : Sets a specific configuration option.

```
waseem.akram082003@gmail.com
[waseemakram@waseem myproject.git % git config
error: no action specified
[waseemakram@waseem myproject.git % git config --list:
error: unknown option 'list:'
usage: git config list [<file-option>] [<display-option>] [--includes]
      or: git config get [<file-option>] [<display-option>] [--includes] [--all] [--regexp] [--value=<pattern>] [--]
      or: git config set [<file-option>] [--type=<type>] [--all] [--value=<pattern>] [--fixed-value] <name> <value>
      or: git config unset [<file-option>] [--all] [--value=<pattern>] [--fixed-value] <name>
      or: git config rename-section [<file-option>] <old-name> <new-name>
      or: git config remove-section [<file-option>] <name>
      or: git config edit [<file-option>]
      or: git config [<file-option>] --get-colorbool <name> [<stdout-is-tty>]

Config file location
  --[no-]global          use global config file
  --[no-]system          use system config file
  --[no-]local           use repository config file
  --[no-]worktree        use per-worktree config file
  -f, --[no-]file <file>
                        use given config file
  --[no-]blob <blob-id> read config from given blob object
```

10. Git Aliases (Custom commands)

→ `git config --global alias.<alias_name>`

`<actual_command>`: Creates a new Git alias for an existing command.

Example: `git config --global alias.co checkout`

```
[waseemakram@waseem myproject.git % git config --global alias.lg "log --oneline --graph"
[waseemakram@waseem myproject.git % git lg
* 7e6c580 (HEAD -> master, tag: v2.0, tag: v1.0, origin/release) Merge branch 'master' into release
| \
| * 2d53ec2 (origin/master, origin/HEAD) added clone of this repo
| * f30fdcd added file1
| /
* bf5dd50 added test from clone
* 874fc01 added two file
waseemakram@waseem myproject.git %
```

11. Git Hooks (Automated scripts triggered by Git actions)

git init: Creates a .git/hooks directory, where hook scripts are stored.

```
waseemakram@waseem myproject.git % git init
[waseemakram@waseem myproject.git % git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /Users/waseemakram/myproject.git/.git/
```

git commit-msg: A hook that runs before a commit message is saved

(you can use this to enforce message formats).

git pre-commit: A hook that runs before a commit is made (can be used for checks like linting).

git post-commit: A hook that runs after a commit is made.

Git pre-commit :~

—>We are creating files in hooks named pre-commit

```
Branch - like set up to track origin/branch -  
waseemakram@waseem myproject.git % vi .git/hooks/pre-commit  
  
waseemakram@waseem myproject.git % chmod +x .git/hooks/pre-commit  
  
waseemakram@waseem myproject.git %
```

```
#!/bin/bash  
echo "PRE-COMMIT HOOK: running before commit..."  
~  
~  
~  
~
```

—>git post-commit:

Same here also we are creating file for post-commit
And adding script to run while commit

```
#!/bin/bash  
echo "POST-COMMIT HOOK: commit completed!"  
~  
~
```

```

branch bike set up to track origin/bike .
[waseemakram@waseem myproject.git % vi .git/hooks/pre-commit

waseemakram@waseem myproject.git % chmod +x .git/hooks/pre-commit

[waseemakram@waseem myproject.git % vi .git/hooks/commit-msg

waseemakram@waseem myproject.git % chmod +x .git/hooks/commit-msg

[waseemakram@waseem myproject.git % vi .git/hooks/post-commit

waseemakram@waseem myproject.git % chmod +x .git/hooks/post-commit

waseemakram@waseem myproject.git % echo "hello world" > file.txt

waseemakram@waseem myproject.git % git add file.txt

waseemakram@waseem myproject.git % git commit -m "test commit"

PRE-COMMIT HOOK: running before commit...
COMMIT-MSG HOOK: checking commit message...
POST-COMMIT HOOK: commit completed!
[bike a628b5f] test commit
1 file changed, 1 insertion(+)
create mode 100644 file.txt
waseemakram@waseem myproject.git % █

```

- > we have created a file after hooks file scripting
 - > after creating file we have added the file and committed it
- We can see after commit

```

PRE-COMMIT HOOK: running before commit...
COMMIT-MSG HOOK: checking commit message...
POST-COMMIT HOOK: commit completed!
[bike a628b5f] test commit
1 file changed, 1 insertion(+)
create mode 100644 file.txt

```

Is displaying after commit

13. Git Workflow Commands (Working with others and managing collaboration)

`git pull --rebase`: Fetches and applies remote changes, rebasing your local commits on top of them.

```
[waseemakram@waseem myproject.git % git pull --rebase origin bike  
  
From github.com:Waseema761/git-document  
* branch      bike      -> FETCH_HEAD  
Current branch bike is up to date.
```

Why use?

- Avoids unnecessary merge commits
- Keeps history cleaner and linear

`git rebase -i <commit_id>`: Starts an interactive rebase to modify commit history (e.g., squash, reword commits)

```
[waseemakram@waseem myproject.git % git rebase -i HEAD~3  
  
[detached HEAD 2e629d1]  added  
Date: Tue Nov 18 19:56:05 2025 +0530  
6 files changed, 6 insertions(+)  
create mode 100644 .gitmodules  
create mode 100644 dog  
create mode 100644 fil7  
create mode 100644 file.txt  
create mode 100644 file4  
create mode 160000 git-practice  
Successfully rebased and updated refs/heads/bike.  
[waseemakram@waseem myproject.git % git log -3  
commit 2e629d1bb49a60d230038e3a1414248d1fff3a5d (HEAD -> bike)  
Author: waseem-akram <waseem.akram082003@gmail.com>  
Date: Tue Nov 18 19:56:05 2025 +0530  
  
    added  
  
    added file  
  
    test commit  
  
commit 466f2854eea3456c2c8f6014e93076c3c70c0272  
Author: waseem_akram <waseem.akram082003@gmail.com>
```

```
pick 381a3a7 # added
squash c8154d6 # added file
squash a628b5f # test commit
```

```
# Rebase 466f285..a628b5f onto 466f285 (3 commands)
```

```
#
```

```
# Commands:
```

```
# p, pick <commit> = use commit
```

```
# r, reword <commit> = use commit, but edit the commit message
```

```
# e, edit <commit> = use commit, but stop for amending
```

```
# s, squash <commit> = use commit, but meld into previous commit
```

```
# f, fixup [-C | -c] <commit> = like "squash" but keep only the previous
#                               commit's log message, unless -C is used, in which case
```

```
#                               keep only this commit's message; -c is same as -C but
```

```
#                               opens the editor
```

```
# x, exec <command> = run command (the rest of the line) using shell
```

```
# b, break = stop here (continue rebase later with 'git rebase --continue')
```

```
# d, drop <commit> = remove commit
```

```
# l, label <label> = label current HEAD with a name
```

```
# t, reset <label> = reset HEAD to a label
```

```
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
```

```
#       create a merge commit using the original merge commit's
```

```
#       message (or the oneline, if no original merge commit was
```

```
#       specified); use -c <commit> to reword the commit message
```

```
# u, update-ref <ref> = track a placeholder for the <ref> to be updated
```

```
#       to this position in the new commits. The <ref> is
```